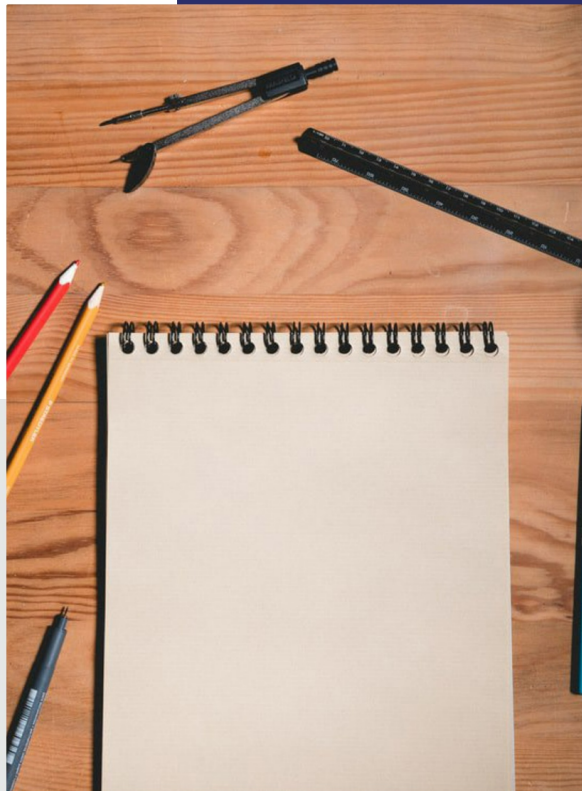




Object-Oriented Drawing Editor in Java

Ali Ayhan Günder

Created using  presentations

Project Overview



A simple object-oriented drawing editor.

An application designed to create and manipulate graphical objects using OOP principles.



Key Technologies: Java, Swing for GUI, AWT for 2D graphics.

Utilizes Java programming language along with Swing for building the graphical user interface and AWT for rendering 2D graphics.



Core Purpose: To illustrate fundamental Object-Oriented Programming (OOP) principles.

The editor serves as an interactive tool for demonstrating essential OOP concepts in a practical setting.

OOP Principles in Action

- **Inheritance allows concrete shape classes to inherit common behaviors from the abstract Shape class.**

Shape (abstract base class) defines common behaviors (draw, contains, move). Concrete shape classes (Circle, Square, Line, StylusPath) inherit from Shape.

- **Polymorphism enables dynamic method execution based on the actual shape type.**

DrawingPanel iterates through a List and calls shape.draw(g2d). The specific draw method executed depends on the actual type of the shape (e.g., Circle.draw for a Circle).

- **Encapsulation controls access to Shape properties through public getters and setters.**

Shape properties (e.g., x, y, color) are protected. Access is controlled via public getters and setters.

- **Abstraction defines a common interface while hiding implementation details.**

The Shape class abstracts the common interface for all drawable objects, hiding implementation details from the DrawingPanel.

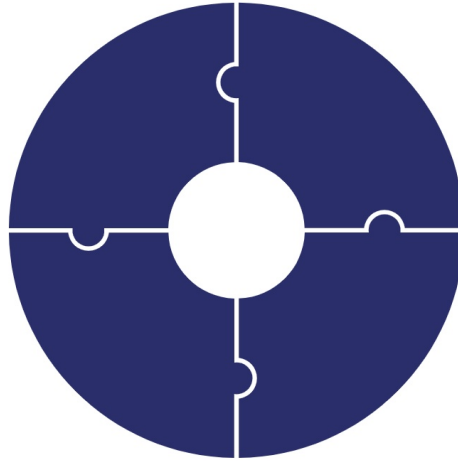
- **Separation of concerns improves code management by dividing responsibilities between components.**

DrawingPanel manages the drawing logic and display. hw1 (main frame) handles UI elements, event listeners, and overall application state.

Key Features

Multiple Drawing Tools: Circle, Square, Line, Stylus (freehand drawing).

Dynamic UI Feedback: Instructions label updates based on selected tool and actions.



Shape Customization: Adjustable Size/Thickness via Slider, Selectable Colors via Dropdown.

Interaction: Shape Selection and Movement (drag & drop), Eraser Tool (click/drag to erase), 'Clean All' button to clear the canvas.

Core Components & Classes



Shape (Abstract Class) defines common attributes and methods.

It includes abstract methods like draw, contains, and move.



Concrete Shape Classes implement specific methods for geometry.

Classes such as Circle, Square, and Line handle their own drawing and selection.



StylusPath manages freehand drawing representation.

It maintains a list of Point objects and checks proximity to path segments.



EraserPath simulates erasing by drawing in the background color.

It extends StylusPath functionality for erasing shapes.

Eraser Implementation Deep Dive

Challenge: Accurately erasing irregular shapes

Ensuring smooth interaction for various shapes such as lines and stylus paths.

Geometric Intersection for Line and StylusPath

Checks intersection using `intersectsLine()` method.

Smooth Visual Feedback with DrawingPanel

Repaint on `mouseMoved` listener for fluid updates of eraser rectangle.



Solution (Improved): Rectangle2D for eraser area

Utilizes geometric representation for effective erasing.

Geometric Intersection for Circle and Square

Checks points on borders and bounding boxes for intersection.

Challenges & Learning

Geometric Hit-Testing

Precisely detecting clicks/erasure on lines and irregular shapes (StylusPath) was a key challenge. Leveraging Line2D.ptSegDist from Java AWT proved crucial for accuracy.

Refactoring

Identifying and extracting 'magic numbers' into constants improved readability and maintainability.



Swing Event Handling

Managing different tool states and mouse interactions (single-click, drag, two-click line) required careful event management.

UI Responsiveness

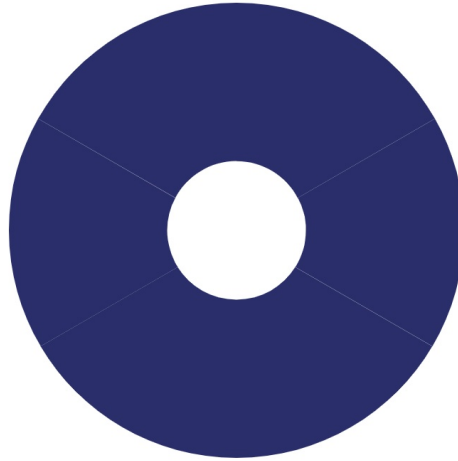
Ensuring the application remains fluid during drawing and dragging operations, especially with many shapes.

Future Enhancements

More complex shapes (e.g., Triangle, Polygon, Text).

Resizing shapes after creation.

More advanced selection (e.g., marquee select).



Undo/Redo functionality.

Save/Load drawings.

Grouping shapes.

Questions & Discussion

Thank you *Please feel free to ask any questions you may have.*